Dokey: A Vietnamese Input Method That Rebuilds the Whole Word on Every Keystroke

Do Huy Toan
Independent researcher, Delaware, USA
ORCID: https://orcid.org/0009-0007-5803-4405 · https://dhtoan.github.io/

Working paper, version 2, November 2026

© 2026 Do Huy Toan. All rights reserved. No part may be reproduced, distributed or adapted in any form without the author's prior written permission. Contact: https://dhtoan.github.io/

Abstract

Vietnamese is typed on standard keyboards through input methods such as Telex and VNI, in which ordinary letter or digit keys add diacritics and tones to the word being typed. Common input methods often produce jumbled or lost letters in browsers and Electron applications, and they add tones to English words. Dokey is a new input method for macOS and Windows, written from scratch, that addresses these problems with three design decisions: (1) the engine keeps only the raw keys of the current word and recomputes the displayed word from them with a pure function on every keystroke; (2) a diacritic or tone key is applied only if the result is still a prefix of a valid Vietnamese syllable, otherwise the keys are shown as typed; and (3) the platform layer applies each correction as a single minimal edit, choosing a delivery strategy per application class. The engine is a dependency-free C++17 library of about 1,300 lines with a C API, used by a Swift/AppKit app on macOS and a Win32 app on Windows. We report the design, the platform layers and an evaluation: all 20,635 automated test cases pass, including a round trip over 3,964 syllables with every tone and a regression corpus of 1,171 English and developer words and 30 Vietnamese sentences; the engine takes a median of 0.7–0.8 µs per key on a 2014 desktop CPU; and 11.9% of keys in the corpus need a correction, with 1.39 deletions per correction on average. We also list what has not been tested, notably real Windows machines and several browsers.

Keywords: Vietnamese; input method; IME; Telex; VNI; keyboard; text input; macOS; Windows

---

1. Introduction

Vietnamese uses the Latin alphabet with additional letters (ă, â, đ, ê, ô, ơ, ư) and five tone marks. On a standard keyboard it is typed through an input method: in Telex, for example, `aa` gives â, `dd` gives đ and a final `s` adds the acute tone, so `tieengs` becomes *tiếng*. The input method intercepts keys, decides how the current word should look, and replaces what is on screen.

Two problems are common in practice. First, the replacement is a sequence of synthetic delete and insert events, and in applications that process keys in a separate process (Chromium-based browsers and Electron

apps) those events can arrive out of order or be dropped, giving jumbled or lost letters. Second, applying tone keys blindly turns English words into nonsense (`windows` → *wíndows*).

Dokey is an input method written from scratch to address both problems. Its first commit is dated 28 October 2026 and version 0.4.0 was released on 1 November 2026. This paper describes its design and reports a reproducible evaluation of the engine. Dokey contains no code from other input methods, and its specification was written from Vietnamese spelling rules and public Telex/VNI conventions.

2. Design

2.1 Raw keys and a pure rebuild

The engine stores the raw key sequence of the word being typed. After every key it rebuilds the displayed word from the whole raw sequence with a pure function: the same raw keys always give the same word. There is no incremental state that can drift from the screen.

The result returned to the platform is an edit: a number of characters to delete and a text to insert, computed from the common prefix of the word currently shown and the new word. If the edit is just "insert the key that was typed", the engine returns *not handled* and the platform lets the original key through, so most keys cause no synthetic events at all.

Word-breaking characters (space, punctuation, Enter, Tab, and digits in Telex mode) end the current word and pass through unchanged.

2.2 Spelling as a gate

A Vietnamese syllable is an initial consonant plus a rhyme, with constraints such as: `k`, `gh`, `ngh` only before e, ê, i, y; `q` always followed by `u`; rhymes ending in c, ch, p, t (and k in Central Highlands place names such as *Đắk Lắk*) take only the acute or dot-below tone. Dokey keeps a dictionary of valid syllables together with all their prefixes.

A diacritic or tone key is applied only if the result is still a prefix of a valid syllable; otherwise the key is printed as a letter. With spelling checks on, if the displayed word stops being a valid prefix, the engine shows exactly the keys typed. English words such as `windows`, `chrome` and `github` therefore stay as typed. Repeating a mark key undoes it (`aaa` → aa), and words of four or more keys that were "undone" this way and are still not Vietnamese are restored in full at the end of the word (`pass`, `error`, `session`).

2.3 Free mark order and tone placement

Because the word is rebuilt from all keys, marks can be typed anywhere in the word: `tieengs`, `tienges` and `tiengse` all give *tiếng*. Tone placement follows written rules: a vowel carrying a diacritic takes the tone (with ươ the tone goes on ơ); otherwise, with a final consonant, the last vowel of the group; in three-vowel groups the middle vowel; in two-vowel groups the first vowel, except that oa, oe and uy take it on the second vowel in the modern style (*hoà*, *khoẻ*, *thuý*) and on the first in the traditional style (*hòa*, *khỏe*, *thúy*). Output is precomposed Unicode (NFC) by default, or combining tone marks.

© 2026 Do Huy Toan. All rights reserved. No reproduction, distribution or adaptation without the author's prior written permission.

2.4 Editing features (version 0.4)

Version 0.4 added: reopening the previous word after one space and Backspace; a Backspace that re-places the tone when deleting a letter moves it; an Esc key that restores the raw keys (`tiếng` → tieesng); "Quick Telex" initial shortcuts (`cc` → ch, `nn` → ng); and user text shortcuts (`vn` → Việt Nam) with case following the abbreviation.

3. Implementation

3.1 Engine

Dokey Core is a C++17 static library with no external dependencies: about 1,300 lines across the engine, the Vietnamese letter and syllable tables, the C API and two public headers. The C API is small: create an engine with options (Telex or VNI, encoding, tone style, spelling check, Quick Telex), pass each key and receive an edit, notify Backspace, reset on context changes, and destroy. A command-line tool converts typed text for testing.

3.2 macOS application

The macOS app (Swift and AppKit, one universal binary for Intel from macOS 10.13 and Apple Silicon from macOS 11) captures keys with a session-level `CGEventTap` at the head of the chain, on its own run-loop thread, and passes the Unicode character of each printable key to the engine. It resets the engine on mouse clicks, arrow and navigation keys, shortcuts with ⌘, ^ or ⌥, key auto-repeat and application switches.

The key design point is delivery. Applications are classified as browsers, Electron/IDE apps or regular apps, by bundle identifier and, for unknown apps, by the frameworks they ship. Regular apps receive Backspace events followed by up to 20 characters per event. Browsers and Electron apps instead receive a selection of the old letters (Shift+← repeated) followed by the replacement text, one UTF-16 unit per event, so that a dropped synthetic Backspace cannot leave stray letters. Synthetic events are tagged so the tap ignores its own output, carry no modifier flags other than the Shift of the selection, and never reuse the code of a key that is still held. Google Docs, Spotlight, Raycast and Alfred get dedicated timings, and a slower profile can be enabled for older machines.

3.3 Windows application

The Windows app (C++ with Win32 and GDI+, builds for x86, x64 and ARM64) uses a low-level keyboard hook (`WH_KEYBOARD_LL`) on its own thread. Each correction is a single `SendInput` call containing the deletions and the Unicode text, so keys typed meanwhile queue behind the correction and cannot overtake it. Chromium and Electron windows (`Chrome_WidgetWin_*`) get a short pause between deletion and insertion. Dokey steps aside while a Chinese, Japanese or Korean IME is active.

3.4 Privacy

Apart from a check for new releases against GitHub's public release API in the macOS app, Dokey makes no network connections. It collects no data and does not store keystrokes; a debug log is off by default and stays

© 2026 Do Huy Toan. All rights reserved. No reproduction, distribution or adaptation without the author's prior written permission.

on the user's machine.

4. Evaluation

4.1 Automated tests

The test suite in the repository was built and run by the author for this paper (repository commit `5f84bd5`; the engine and tests are unchanged since the version 0.4.0 tag; clang 21.1 at `-O2` on Windows 11, Intel Core i5-4590). The same suite runs on Linux, macOS and Windows in continuous integration after every change.

Test group	Cases	Result
Round trip: every syllable typed with every tone, then compared with the expected word	3,964 syllables × tones	0 failures
English and developer words that must stay as typed	1,171 words (1,097 kept as typed; 74 are valid Vietnamese syllables and are expected to change)	0 failures
Real Vietnamese sentences typed in Telex	30 sentences	0 failures
Editing features, C API and pass-through checks	—	0 failures
Total	**20,635 checks**	**20,635 passed**

4.2 Engine cost per key

A small benchmark (kept with this paper) feeds the repository's own corpus, the 30 Telex sentences followed by the English word list (9,389 keys per pass), through the C API 200 times per run, with spelling checks and the modern tone style on. Seven runs were made per invocation, and the program was invoked twice.

Measure	Result
Median time per key (two invocations)	803 ns and 689 ns
Range across the 14 runs	659–900 ns
Keys that needed a correction (edit sent)	11.9%
Deletions per correction (mean)	1.39

Under a microsecond per key on a 2014 desktop processor is far below the interval between keystrokes of a fast typist, so the full rebuild on every key costs nothing noticeable. Because 88% of keys pass through untouched and corrections delete on average fewer than one and a half characters, the platform layer has little to send, which reduces the chance of reordering in browsers.

4.3 Manual typing tests

The project documents its manual tests honestly, including failures:

- On macOS 14 (Intel), version 0.3.0 typed correctly in TextEdit, Terminal, and text areas in Microsoft Edge at 8–120 ms per key, but failed in a ProseMirror editor, which ignored synthetic Backspace events

© 2026 Do Huy Toan. All rights reserved. No reproduction, distribution or adaptation without the author's prior written permission.

and produced *Hayãy*. Version 0.3.1 introduced select-and-replace delivery for browsers; replaying the same event sequence then gave the correct result, and in ProseMirror 6 of 7 runs were fully correct, the failure being the first word right after page load.
- Fast typing in which a vowel key was still held when the tone key arrived (`Haxy`) lost letters until synthetic events stopped reusing held key codes.
- On Windows, simulated typing into Notepad at 40 ms per key and at full speed returned the expected text, on GitHub Actions virtual machines (Windows Server 2022, x64).

5. Limitations

- The Windows app has not yet been run on a real Windows PC; its tests ran on virtual machines, and the x86 and ARM64 builds have only been compiled.
- Typing has not been tested directly in Chrome or VS Code, on macOS 10.13–10.15, on Apple Silicon hardware, or on slow machines; the delivery pauses are initial estimates.
- In Electron terminals (for example the VS Code terminal) Shift+← does not select text, so tones can be placed wrongly there. With combining Unicode output, Backspace delivery is still used.
- When an application enables secure input (password fields, Terminal's Secure Keyboard Entry), macOS blocks every input method by design.
- The releases are not yet code-signed. There is no user study, and no comparison with other input methods was made for this paper.

6. Conclusion

Rebuilding the whole word from raw keys on every keystroke, gating marks by syllable validity, and sending each correction as one minimal edit chosen per application class give a Vietnamese input method whose screen state cannot drift from its internal state and which leaves English words alone. The engine is small, dependency-free and fast enough that the full rebuild is free in practice. The main open work is testing on real Windows machines and more browsers, and adding legacy encodings (TCVN3, VNI Windows, CP1258).

Software availability

Dokey is proprietary software, © 2026 Do Huy Toan, all rights reserved. Source and releases: https://github.com/dhtoan/Dokey (version 0.4.0, 1 November 2026).

Acknowledgements and AI-use statement

The author's company is a member of Anthropic's Claude Startups program and received Claude Team access and API credits through it. Claude (Anthropic) assisted with code, debugging and the drafting of this paper. In the author's own experience it made project work roughly ten times faster: problems that once ended with a long sip of water and a longer stare at the screen now usually end with a working fix. The author reviewed all content and is solely responsible for it; Anthropic did not sponsor, review or endorse this work.

References

© 2026 Do Huy Toan. All rights reserved. No reproduction, distribution or adaptation without the author's prior written permission.

[1] Dokey source repository and release notes. https://github.com/dhtoan/Dokey [2] Apple. Quartz Event Services (CGEventTap). https://developer.apple.com/documentation/coregraphics/quartz_event_services [3] Microsoft. LowLevelKeyboardProc (WH_KEYBOARD_LL) and SendInput. https://learn.microsoft.com/windows/win32/winmsg/lowlevelkeyboardproc · https://learn.microsoft.com/windows/win32/api/winuser/nf-winuser-sendinput [4] Unicode Consortium. Unicode Normalization Forms (UAX #15). https://unicode.org/reports/tr15/

© 2026 Do Huy Toan. All rights reserved. No reproduction, distribution or adaptation without the author's prior written permission.